

LLM Training with LoRA - Continued Pre-training

Assignment Report

Course: LLM Models - BGU

Task: Adapt a pre-trained language model (Qwen2.5-1.5B) to the Sherlock Holmes corpus using LoRA, evaluate via perplexity, and run ablation experiments.

GitHub Repository: <https://github.com/RonBeiden/LLM-LoRA-Pretraining>

1. Model Choice & Memory Estimation

Selected model: Qwen/Qwen2.5-1.5B - a 1.54B parameter model with GQA and a large vocabulary (151K tokens).

Memory Estimation Summary (from Notebook 01)

Model	Params	Weights FP16	Weights NF4	Full-Precision Train	QLoRA Train	Total QLoRA (S=512)	Fits 11GB
Qwen2.5-1.5B	1.54B	2.9 GB	0.7 GB	22.9 GB	1.6 GB	2.6 GB	✓
Llama-3.2-3B	3.21B	6.0 GB	1.5 GB	47.8 GB	3.3 GB	4.6 GB	✓
Llama-3.1-8B	8.03B	15.0 GB	3.7 GB	119.7 GB	8.2 GB	10.3 GB	✓ (tight)
Llama-2-13B	13.02B	24.3 GB	6.1 GB	194.0 GB	13.3 GB	15.8 GB	X

Rationale for choosing 1.5B: Fits comfortably on our 6GB RTX A1000 laptop GPU with QLoRA + gradient checkpointing. Leaves headroom for batch size > 1 and experimentation.

2. Data

Corpus

The 9 canonical Sherlock Holmes books by Arthur Conan Doyle, downloaded from Project Gutenberg.

Key Statistics

- **Total tokens:** ~895,000 (Qwen2.5-1.5B tokenizer)
- **Training tokens:** ~838,000 (8 books)
- **Validation tokens:** ~57,000 (1 book - *The Sign of the Four*)
- **Validation split:** 6.4% - a complete novel held out for coherent perplexity measurement
- **Training chunks:** 1,636 samples × 512 tokens each
- **Validation chunks:** 112 samples × 512 tokens each

Preprocessing Steps

1. Downloaded 9 books from Project Gutenberg
2. Stripped Gutenberg headers/footers (regex-based detection of START/END markers)
3. Cleaned special characters: normalized Unicode (curly quotes → straight, em-dash → --, etc.), removed `_emphasis_` markers, collapsed multiple blank lines
4. Contiguous chunking: tokenized full training text, split into non-overlapping chunks of 512 tokens
5. Discarded final incomplete chunk

Chunking Strategy

We use **contiguous (non-overlapping) chunking** - the full training corpus is concatenated and split into fixed 512-token windows. This preserves natural text flow for causal language modeling without wasting compute on overlapping tokens. TRL's `packing=True` further optimizes by combining shorter sequences.

Input Length Analysis

The paragraph-level analysis (see Notebook 02) shows that 99%+ of paragraphs fit within 512 tokens. Only rare passages exceed 1024 tokens. Our choice of `max_length=512` covers the corpus well while keeping memory manageable.

3. Training Configuration

LoRA Configuration

Parameter	Value
Rank (r)	16
Alpha	32
Dropout	0.05
Target modules	q_proj, v_proj
Quantization	4-bit NF4 (QLoRA)
Double quantization	Yes

Parameter	Value
Compute dtype	BFloat16

Training Arguments

Parameter	Value
Epochs	3
Learning rate	2e-4
LR scheduler	Cosine
Warmup	5% of steps
Batch size	2 (per device)
Gradient accumulation	8 (effective batch = 16)
Weight decay	0.01
Gradient checkpointing	Yes
Packing	Yes
Optimizer	AdamW (8-bit via bitsandbytes)

Baseline Training Results

- **Final training loss:** 2.554
- **Training time:** ~57 minutes
- **Peak GPU memory:** 3.89 GB / 6.0 GB available
- **Total steps:** 309

4. Evaluation Results

Perplexity (Sliding Window, stride=256)

Model	Perplexity
Base (Qwen2.5-1.5B, no training)	14.49
Adapted (LoRA r=16, q/v)	13.65
Improvement	5.8%

The LoRA-adapted model achieves a meaningful reduction in perplexity on the held-out Sherlock Holmes text, confirming that domain adaptation occurred.

Learning Process Monitoring

- Training loss was logged every 10 steps to TensorBoard (`report_to='tensorboard'`)
- Evaluation loss was computed every 100-200 steps on the validation set
- Loss curves show steady convergence with no signs of divergence (see Notebooks 03 and 05)

5. Experiments & Ablations

We ran **4 ablation experiments** (minimum required: 3):

Ablation 1: LoRA Rank ($r=4$ vs $r=16$ vs $r=64$)

- **Hypothesis:** Higher rank $\rightarrow$ more capacity $\rightarrow$ lower perplexity, with diminishing returns
- **Results:** $r=4$: 13.77, $r=16$: 13.64, $r=64$: 13.54
- **Conclusion:** Confirmed. Monotonic improvement with diminishing returns. $r=16$ captures most of the benefit.

Ablation 2: Learning Rate ($1e-5$ vs $1e-4$ vs $5e-4$ vs $1e-3$)

- **Hypothesis:** Very low LR under-trains; very high LR destabilizes
- **Results:** $1e-5$: 14.33, $1e-4$: 13.75, $5e-4$: 13.58, $1e-3$: 13.69
- **Conclusion:** Sweet spot at $5e-4$. LR= $1e-3$ shows mild overfitting (lowest train loss but worse PPL than $5e-4$). LR= $1e-5$ barely adapts.

Ablation 3: Target Modules ($\{q,v\}$ vs $\{q,k,v,o\}$ vs all linear)

- **Hypothesis:** More modules $\rightarrow$ more capacity, with diminishing returns from FFN layers
- **Results:** q,v : 13.64, q,k,v,o : **13.41** (best overall), all linear: 13.46
- **Conclusion:** Confirmed. q,k,v,o is optimal - FFN layers add parameters but slightly overfit on this small corpus.

Ablation 4: Warmup & Scheduler (constant vs cosine $\times$ warmup vs no warmup)

- **Hypothesis:** Warmup + cosine would be best
- **Results:** no_warmup+constant: 13.59, warmup+constant: 13.62, no_warmup+cosine: 13.65, warmup+cosine: 13.64
- **Conclusion:** Surprisingly, differences are minimal (spread: 0.06 PPL). Constant LR slightly outperforms cosine for this short training run. Warmup has negligible effect with LoRA (adapters start near zero).

Best Configuration Found

Parameter	Optimal Value
LoRA rank	64 (or 16 for efficiency)

Parameter	Optimal Value
Learning rate	5e-4
Target modules	q, k, v, o
Scheduler	Constant (no decay)
Best PPL achieved	13.41 (q,k,v,o config)

6. GPU & Hardware Used

Hardware	Specs	Used For
NVIDIA RTX A1000 Laptop GPU	6 GB VRAM, CUDA 12.8	All training & evaluation (notebooks 03-05)
CPU	Intel (local machine)	Notebook 01 (memory estimation), Notebook 02 (preprocessing)

Note: We initially attempted to use the department lab server (RTX 2080 Ti) for faster training, but the remote server was intermittently unavailable. All final results were produced on the local RTX A1000 6GB laptop GPU, which required careful memory management (batch_size=2 + gradient_accumulation=8).

We did **not** use the BGU cluster queue (`--partition=rtx2080`) since the local GPU was sufficient for the 1.5B model with QLoRA.

7. Assumptions

- Validation split:** We chose *The Sign of the Four* (6.4% of corpus) as the held-out validation set. This is a complete novel, ensuring perplexity is measured on coherent, extended text rather than random snippets.
- Perplexity computation:** We use sliding-window perplexity with `max_length=512` and `stride=256` , following the HuggingFace guide. Only non-overlapping tokens are scored to avoid double-counting.
- Quantization impact:** We assume NF4 quantization (QLoRA) has negligible impact on final quality for a 1.5B model. Base perplexity is measured with the same quantization for fair comparison.
- Seed determinism:** All experiments use `seed=42` . Due to GPU non-determinism in some CUDA kernels, results may vary slightly across runs, but relative comparisons remain valid.
- Packing:** We enable TRL's `packing=True` which concatenates multiple samples into single sequences for efficiency. This means attention can "bleed" across sample boundaries (no flash attention on this GPU), but for continued pre-training on a single-domain corpus this is acceptable.

6. **Tokenizer:** We use the model's native Qwen tokenizer without modification (vocab size 151,936). No special domain tokens were added.
7. **Experiment fairness:** All ablation experiments train for exactly 3 epochs with the same effective batch size (16) and data. Only the varied hyperparameter differs.

8. What Did Not Work - Mistakes Log

1. Remote Server Crashes

- **Problem:** Initially tried to run notebook 04 (evaluation) on a remote server (RTX 2080 Ti) for faster training. The server went offline mid-training.
- **Solution:** Fell back to local GPU. Added a **checkpoint/resume system** to notebook 05 that saves `results.json` + `perplexity.json` after each experiment, so crashed sessions can resume without re-training. This saved hours when the laptop was accidentally moved during ablation 4.

2. Windows Encoding Crash (TRL Jinja Templates)

- **Problem:** TRL's internal Jinja template files (e.g., `deepseekv3.jinja`) contain non-ASCII characters. On Windows, Python's `pathlib.Path.read_text()` defaults to `cp1252` encoding, causing `UnicodeDecodeError`.
- **Solution:** Monkey-patched `pathlib.Path.read_text` at the top of each notebook to default to UTF-8 encoding. This is applied before importing TRL.

3. API Changes Between Package Versions

- **Problem:** Several parameters were deprecated or renamed in the latest versions:
 - `torch_dtype` → `dtype` in transformers 5.x
 - `max_seq_length` → `max_length` in TRL 1.4.0
 - `trust_remote_code=True` no longer needed for Qwen2.5 (natively supported)
- **Solution:** Updated all calls to use the new API names after reading deprecation warnings.

4. OOM with Batch Size 4

- **Problem:** Initial configuration used `batch_size=4` (from RTX 2080 Ti setup). This caused OOM on the 6GB RTX A1000.
- **Solution:** Reduced to `batch_size=2` with `gradient_accumulation_steps=8` to maintain effective batch size of 16.

5. Kernel Death During Experiments

- **Problem:** Moving the laptop during ablation 4 training killed the Jupyter kernel, losing all in-memory results.

- **Solution:** Implemented disk-based checkpointing - `is_experiment_complete()` checks for saved results before re-running, and `evaluate_perplexity_cached()` caches PPL results to JSON files.

6. Packing Warning (No Flash Attention)

- **Problem:** TRL warns about cross-contamination between packed samples when not using `flash_attention_2/3`. Our GPU doesn't support FlashAttention.
- **Impact:** Minimal for continued pre-training on a single-domain corpus (all samples are from the same Sherlock Holmes text). For SFT in Part 2, this may need revisiting.
- **Kept:** Packing enabled for efficiency; accepted the theoretical cross-contamination risk.

9. AI Assistant Usage

Tool used: GitHub Copilot (Claude-based agent in VS Code)

The AI assistant was used extensively throughout this assignment for:

1. **Notebook scaffolding:** Generated the initial structure for all 5 notebooks, including markdown explanations, code cells, and plotting logic. Each notebook was then iteratively refined based on execution results.
2. **Debugging runtime errors:** The assistant diagnosed and fixed several issues that arose during execution - Windows encoding issues, API deprecation changes, OOM errors - providing immediate fixes rather than requiring manual Stack Overflow searches.
3. **Checkpoint/resume system:** After a kernel crash during experiments, the assistant designed and implemented a disk-based caching system that allows experiments to be resumed without re-training.
4. **Running experiments end-to-end:** The assistant executed all notebook cells sequentially, monitoring outputs for errors and adjusting parameters (e.g., batch size reduction) when issues arose.
5. **Analysis writing:** The hypothesis reconciliation text in notebook 05 was written by the assistant based on the actual numerical results, then reviewed for correctness.

What the AI did NOT do: The conceptual decisions (model choice, ablation selection, validation split strategy) were specified by the student. The AI implemented these decisions and handled the engineering complexity.

10. Summary & Conclusions

Key Findings

- 1. **LoRA effectively adapts a general LLM to a literary domain:** Even with minimal compute (6GB GPU, 57 min training), we reduced perplexity from 14.49 to 13.65 (5.8% improvement) on held-out Sherlock Holmes text.
- 2. **Target modules matter most:** The biggest single improvement came from expanding LoRA to all attention projections (q,k,v,o), achieving PPL=13.41 - a 7.5% improvement over the base model.
- 3. **Scheduler choice barely matters for short runs:** All 4 scheduler configurations produced nearly identical results (spread of 0.06 PPL). For 3-epoch training on a small corpus, the learning rate value matters far more than the schedule shape.
- 4. **QLoRA makes large-model fine-tuning accessible:** A 1.5B model required only 3.89 GB peak memory with QLoRA, fitting easily on a laptop GPU. Without quantization, the same training would need ~23 GB.

Final Results Table

Experiment	Perplexity	Final Loss
Base model (no training)	14.49	-
Baseline (r=16, q/v, lr=2e-4, cosine)	13.65	2.554
Best rank (r=64)	13.54	2.519
Best LR (5e-4)	13.58	2.520
Best modules (q,k,v,o)	13.41	2.517
Best scheduler (constant, no warmup)	13.59	2.544